

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

MOHSIN IBNA HOSSAIN
AIUB, DATA BASE NOTES

DATA
BASE

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : ____ / ____ / ____

Table of Contents

SL.NO	Topic	Pages
1	Introduction to Database	03-05
2	Data Type	05
3	How to Create, Insert, Delete, Rename a Table	06-15
4	Keys	15
5	Constrains	16-19
6	DISTINCT & SORTING	20-23
7	Arithmetic, Comparison & Logical Operator	23-25
8	Where Clause	26-27
9	Use of Relational & Logical Operators	27-32
10	Types of Functions	33-52
11	GROUP BY & HAVING	53-54
12	How to Add, Drop, Modify a Column from a Table	55-57
13	How to Add, Drop, Disable & Enable Constraints	59-61
14	ER Diagram	62-85

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

DATA BASE! -

⇒ Data Base is a collection of data in a format that can be easily accessed.

DBMS! -

⇒ A software application used to manage our DB is called DBMS (Database Management system).

Types of Databases! -

Relational! -

⇒ Data stored in tables.

Non-Relational (No SQL)! -

⇒ Data Not stored in tables.

#SQL: (Structured Query Language)

Q. What is SQL?

⇒ SQL is a programming language used to interact with relational Database.

It is used to perform CRUD operations:-

1. Create

2. Read

3. Update

4. Delete.

How to Create A Database: (for MySQL)

Query: Syntax:-

CREATE DATABASE db_name;

Ex:-

CREATE DATABASE temp1;

OR

create database temp1;

Q:- Create a Database For a collage?

CREATE DATABASE collage;

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

How to DROP a Database! (For MySQL)

⇒ DROP means Delete.

Query Syntax! —

DROP DATABASE db-name;

Ex:- DROP DATABASE temp1; or

⇒ Drop database temp1;

Data Type! —

1. Numeric Data type! —

1. int

2. float(size)

3. NUMBER(size)

4. NUMBER(m,D)

↓
Represent
Total Digit in
the float
Number

↑
represent
total Digit
After the
decimal
point.

2. Character Data type! —

1. CHAR(20) → For a fixed length string with exactly 20 char.

2. VARCHAR(50) → For string maximum length of 50 char.

↓
represent
the length of the
string.

How to Create a Table:-

Query Syntax:-

```
CREATE TABLE table-name(  
    column-Name1 datatype(size),  
    column-Name2 datatype(size),  
    column-Name3 datatype(size),  
    ....  
    column-name datatype(size)  
);
```

Q:- Create a table where columns will be Name, Id, CGPA.

```
=> CREATE TABLE student(  
    name varchar2(50),  
    Id varchar2(10),  
    CGPA NUMBER(3,2)  
);
```

Q:- Create a table where columns will be Name, Roll, Gender, Age, GPA, City.

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

⇒ CREATE TABLE Student (

Name varchar2(50),

Roll number(5),

Gender varchar2(6),

Age int,

CGPA number(9, 2),

City varchar2(50)

);

How to Renaming Tables:—

Query Syntax:—

ALTER TABLE old-name RENAME TO new-name;

Qr Change the name of the previous "Student" table.

⇒ ALTER TABLE Student RENAME TO StudentTable;

How to Delete A Table:—

Query Syntax:—

DROP TABLE table-name;

Q:- Now Delete/Drop the studentTable.

⇒ DROP TABLE studentTable;

How to view All columns
of a table without inserting any Data.
OR, How to view the descriptions of
a Table.

Query Syntax:-

1. DESCRIBE table-name; or

2. DESC table-name;

Q:- Show the description of studentTable and Create a table where columns will be Name, ID, CGPA. then Show the description of that table.

⇒ DESCRIBE studentTable;

```
CREATE TABLE student(  
    Name varchar2(50);  
    ID    varchar2(10);  
    CGPA  number(3,2)  
);
```

DESC student;

How to Insert Data In a table:—

Quarry Syntax:-

① INSERT INTO *table name* (column1, column2, ..., columnN)
VALUES (value1, value2, ..., valueN);

Ex:-

Q:- Now Insert the data for a student in the previous Student table.

```
⇒ INSERT INTO student (NAME, ID, CGPA)
VALUES ('Ali', '23-50186', 9.88);
```

Quarry Syntax:

②. INSERT INTO *table Name*
VALUES (value₁, value₂, ..., value_N);

Ex:-

Q:- Now Insert Another data for a student in the previous Student table.

```
⇒ INSERT INTO student+  
VALUES ('JAM', '23-80187', 9.78);
```

Note:- યાદ રાખો Data type Varchar/Char જરૂર - કાળે
અને Value Input નો - માન્ય જરૂર થાય.

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

How to View the Data from Table!—

☞ If we want to view all the Data from a table!—

Query Syntax!—

SELECT * FROM tableName;

Ex:-

☞ Show all data associated in student table.

⇒ SELECT * FROM student;

(represent All Column in a table)

// For output:—

☞ If we want to view just a Column data from a table!—

Query Syntax—

SELECT ColumnName from tableName;

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Q:- Show the Name Data associated in student table.

⇒ `SELECT Name FROM student;`

// For output

If we want to view more than 1 column data from a table:-

Q:- Show the name, CGPA Data associated in student table.

⇒ `SELECT Name, CGPA`

`FROM student;`

// For output

practice Qs: 1

Step 1:- Create a table, inside the table store employee info (id, name and salary).

Step 2:- Add following information in the table:

1. 'MOHSIN', 23, 105000

2. 'Jam', 24, 100000

3. 'Arif', 25, 100000

Step 3:- Select and view all the table Data.

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

⇒ CREATE TABLE empInfo (

Name VARCHAR2(50);

ID VARCHAR2(10);

Salary number(10)

);

⇒ INSERT INTO empInfo

VALUES ('MOHSTIN', '22-50184-1', 1000000);

INSERT INTO empInfo

VALUES ('JAM', '22-50187-1', 1000000);

INSERT INTO empInfo

VALUES ('ALIF', '22-50186-1', 1000000);

⇒ SELECT * FROM empInfo;

// For (Output in screenshot)

How to view the Limited
Number of Rows/Data from
a table.

Query Syntax:-

SELECT *

FROM (SELECT table-name.*, ROWNUM
FROM table-name)

WHERE ROWNUM $\leq$ N;

N = No. of Row

We want to
view

Q. Show only first two persons data associated in empInfo Table.

⇒ SELECT *

FROM (SELECT empInfo.*, ROWNUM
FROM empInfo)

WHERE ROWNUM $\leq$ 2;

For output

How view Limited Number of
rows from a table between
m to n

Query Syntax:-

```
SELECT *  
FROM ( SELECT tableName.*,  
        ROWNUM AS rename-the-ROWNUM  
        FROM tableName)  
WHERE rename-the-ROWNUM
```

```
BETWEEN m AND n;  
         ↓      ↓  
Starting Point Ending Point
```

Q:- Show Data of persons from 2 to 4 rows from empInfo table.

```
⇒ SELECT *  
   FROM (SELECT empInfo.*,  
               ROWNUM AS rnum  
          FROM empInfo)  
  WHERE rnum  
 BETWEEN 2 AND 4;
```


TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

// For Output.

#keys:

Primary key $\Rightarrow$

$\Rightarrow$ It is a column (or set of columns) in a table that uniquely identifies each row. (a unique id) there is only pk and it should be NOT NULL.

Foreign key $\rightarrow$

$\Rightarrow$ A Foreign key is a column (or set of columns) in a table that refers to the primary key of another table. there can be multiple Fks.

Fks have duplicate & null values.

// Example

Constraints:-

⇒ SQL constraints are used to specify rules for data in a table.

① "NOT NULL" Constraints:-

⇒ columns cannot have a null value.

Ex:- col1 int NOT NULL

② "UNIQUE" Constraints:-

⇒ All values in column are different.

Ex:-

col2 int UNIQUE

③ "PRIMARY KEY":-

⇒ makes a column unique & not null but only for one.

Ex:- id int PRIMARY KEY

④ "FOREIGN KEY":-

⇒ prevent actions that would destroy links between tables.

Ex:- CREATE TABLE temp (

cust_id int,

FOREIGN KEY (cust_id) references customers(id)

);

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Q:- Create a table, Inside the table stone productInfo
(Product ID should be primary key, product serial
should be Unique, product Name should be Not null)

⇒ Product Table:-

CREATE TABLE productInfo (

ID NUMBER : (11)

PRIMARY KEY (ID),

Serial NUMBER UNIQUE,

Name VARCHAR2(50)

);

There is 2 Approach to

Declared a primary key:-

Query Syntax:-

1st:- ID NUMBER PRIMARY KEY;

2nd:- ID NUMBER;
PRIMARY KEY (ID);

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

There is 2 Approach to

Declare a "Unique" Constraints:-

Query Syntax:-

1st:- ID NUMBER UNIQUE ;

2nd:- ID NUMBER
UNIQUE (ID) ;

DEFAULT Constraints:-

⇒ Sets the default value of a column:

Example:-

Salary INT DEFAULT 25000 ;

Query Syntax:- with Ex:-

CREATE TABLE temp1

ID int,

salary int DEFAULT 25000 ;

INSERT INTO temp1

(ID) VALUES (23501041) ;

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

"Check" Constraints:-

⇒ It can limit the values allowed in a column.

Query Syntax with Em:-

```
CREATE TABLE temp1 (
```

```
  city VARCHAR2(50) CHECK (city='Dhaka'),
```

```
  id NUMBER,
```

```
  salary int DEFAULT 25000,
```

```
  age int CHECK (age >= 18)
);
```

```
INSERT INTO temp1
```

```
  (city, id, age) VALUES ('Dhaka', 23501041, 18);
```

```
SELECT * FROM temp1;
```

// output

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

// output

"output"

#SORTING:-

⇒ Sorting is a process of rearranging data into meaningful Order so that we can Analyze it more effectively.

There are two type of Sorting

- ① Ascending Order. → (बढ़ते-बढ़ते)
- ② Descending Order. → (घटते-घटते)

Query Syntax:-

SELECT Column-Name

FROM Table-Name

ORDER BY Column-Name (Asc/Desc);

Q:- Show the ename, salary and possible jobs from the emp table eliminating the duplicate output. Also show the salary in reversed order.

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

⇒ SELECT DISTINCT Job, ename, sal
FROM emp
ORDER BY sal DESC;

// Output

Note :- With Order by - it is column name in
Order by - it type is int. Default - Ascending
Order - output 3/4.

Q:- Show the ename, job and sal of the employees
according to the Order (default) to the salary table.

⇒ SELECT ename, job, sal
FROM emp
ORDER BY sal;

// Output

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Multiple Column Sorting :-

⇒ When we want to sort data by multiple column then we use **ORDER BY** clause. **Column, Column, Column, ...**

Query Syntax :-

```
SELECT Column1, Column2, Column3  
FROM Table Name  
ORDER BY Column, Column3 DESC
```

Arithmetic Operators :-

'+' = is used to add two or more variable.

'-' = is used to subtract

'*' = is used to multiplication.

'/' = is used to obtain the quotient. (division)

'%' = is used to get fraction (division).

Comparison Operator:

- '=' $\Rightarrow$ Used to test for equality between two expressions.
- '<=' $\Rightarrow$ Used to test compare two expressions.
- '>' $\Rightarrow$ Used to compare values in a database and retrieve rows where a specific column's value is greater than a given criteria.
- '<' $\Rightarrow$ Used to compare values that are less than a particular value.
- '>=' $\Rightarrow$ Used to compare values that are greater than or equal a particular value.
- '<=' $\Rightarrow$ Used to compare values that are less than or equal a particular value.
- 'BETWEEN' $\Rightarrow$ Used to test whether an expression is within a range of values.

Logical Operator:

'AND' → Combines two condition and returns TRUE if both condition are TRUE. Otherwise returns FALSE.

'OR' → Combines two condition and returns TRUE if at least one condition is TRUE. If both condition are False then returns False.

'IN' → TEST for the truth of some condition.

'NOT' → We can put before any conditional statement to select rows for which that statement is FALSE.

Note:- NOT is commonly used with LIKE.

'LIKE' → Used to determine whether a specific character string matches a specified pattern.

#WHERE clause:-

⇒ ଏହା ଟାବଲ୍‌ରୁ ଡାଟାକୁ ଏକାଠି ମିଳିଥିବା ଷର୍ତ୍ତ/Condition ଉପରେ ଆଧାରିତ ଭାବରେ ଡାଟାକୁ ଫିଲ୍ଟର କରିଥାଏ।

Query Syntax:-

```
SELECT column list
FROM Table Name
WHERE Condition;
```

Q:- Show the ename and job where sal is not equal to 1200 from emp table.

```
SELECT ename, job
FROM emp
WHERE sal <> 1200;
```

output

Q:- show the ename, job, hiredate and empno for the people whose job is 'MANAGER' from the

```
SELECT ename, job, hiredate, empno
FROM emp
WHERE job = 'MANAGER';
```

// output.

Q:- Show ename, job, sal when the sal is greater than Commission and show the sal in descending order.

```
SELECT ename, job, sal
FROM emp
WHERE sal > comm
ORDER BY sal desc;
```

// output.

Use of Relational Operators:-

BETWEEN Operator:-

Q. Show the empno, job when the salary is in range (1200-4000). From the emp table.

```
SELECT empno, job, sal
FROM emp
WHERE sal
BETWEEN 1200 AND 4000
```

// output

Use of Logical operators:-'OR' - operators:-

Q. FROM the emp table show the ename, job, of the employees where the ^{job}~~name~~ name is 'SALESMAN' OR salary greater than 2000.

⇒
SELECT ename, job, sal
FROM emp
WHERE : job = 'SALESMAN' OR sal > 2000;

// Output

'AND' - operator:-

Q. FROM the emp table show the ename, job, of the employees where the job name is 'SALESMAN' AND salary greater than $2000/2 = 1000$.

SELECT ename, job, sal
FROM emp
WHERE job = 'SALESMAN' AND sal > 1000;

BELOW output

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

AND, OR - Combined Operation

Q. From the emp table show the ename, job, of the employees where the empno greater than 7000 AND job name is 'SALESMAN' OR sal greater than 1600

```
=> SELECT ename, job, sal, empno
      FROM emp
      WHERE empno > 7000 AND
            (job = 'SALESMAN' OR sal > 1600);
```

// output

IN - Operators

Q:- From the emp table show the ename, job, of the employees where the job is 'SALESMAN', 'CLERK', 'MANAGER'

```
SELECT ename, job
      FROM emp
      WHERE job IN ('SALESMAN', 'CLERK', 'MANAGER');
```

// output

'NOT' - Operator:-

Q:- Show the ename and job where job is not equal to 'SALESMAN', 'CLERK', 'MANAGER'

⇒ SELECT ename, job.
FROM emp
WHERE job NOT IN ('SALESMAN', 'CLERK', 'MANAGER');

// output

Note:- 'Not' - operator is used with AND, OR, etc. words together.

'LIKE' - Operator:-

⇒ When we use this operator with a table name, then we use the like operator to search for a pattern. It is used to find the pattern in the string and output the result.

Q:- Show the employee names where they have 'T' letter in their name anywhere. From the emp table.

⇒ SELECT ename
FROM emp
WHERE ename LIKE '%T%';

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Output

Q:- Show the employee names where they have 'T' letter in there name in the 1st position.

```
=> SELECT ename
FROM emp
WHERE ename LIKE 'T%';
```

Q: Show the emp. Names where they have 'T' letter in there name 2nd Last position.

```
SELECT ename
FROM emp
WHERE ename LIKE '%T_';
```

// output

Q:- Show the emp names where they have 'U' letter in these names in the 2nd position.

```
SELECT      ename  
FROM        emp  
WHERE ename LIKE '_U%' ;
```

// output .

Q:- Show the emp names where they have 'T' letter in these name in the last position.

```
SELECT      ename  
FROM        emp  
WHERE ename LIKE '%T' ;
```

// output .

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

'AS' keyword for

Custom names:—

⇒ 'AS' keyword යනු නව නමක් ලෙස නම් කිරීම සඳහා යොදා ගන්නා අතර column හි Custom නමක් ලෙස නම් කිරීම සඳහා යොදා ගනු ලබයි.

Query Syntax:—

```
SELECT column Name AS new-column name  
FROM table-name;
```

Q:- Show the column `ename` as `EmpName` From the table of `emp`.

⇒

```
SELECT ename AS EmpName  
FROM emp;
```

// output—

Upper functions:-

⇒ যদি আমরা কোনো টেবিল-এর কোনো String-এ বা Column-এ String value-এর সব বড় শব্দকে প্রদত্ত করে তাহলে আমরা এই function-টি প্রয়োগ করি।

Query Syntax:-

```
SELECT UPPER(ename), ename
FROM emp;
```

Lower functions:-

⇒ যদি কোনো টেবিল-এর কোনো Column-এ String value-এর সব ছোট শব্দকে ছোট করে প্রদত্ত করে তাহলে আমরা Lower() function-টি প্রয়োগ করতে পারি।

Query Syntax:-

```
SELECT LOWER(ename), ename
FROM emp;
```

// output .

Initcap Function:-

⇒ Replace the first letter of every word in a character string with an uppercase letter.

Query Syntax:-

```
SELECT INITCAP(ename || ' ' || job)
FROM emp;
```

Character ManipulationFunction:-

1. CONCAT('Good', 'String') → GoodString

2. SUBSTR('String', 1, 3) → Str

3. LENGTH('String') → 6

4. INSTR('String', 'r') → 3

5. TRIM('s' FROM 'SSMITH') → MITH

6. REPLACE('toY', 'Y', 'let') → tolet.

Number Functions:-

There are three types.

1. Round(),

2. Trunc(),

3. Mod(),

ROUND Function:-

⇒ Return Rounds value to specified decimal

Query Syntax:

```
SELECT ROUND(45.932, 2),
       ROUND(45.932, 0),
       ROUND(45.932, -1)
```

FROM DUAL;

Output:-

P₁
45.93

P₂

46

P₃

50

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

TRUNC Functions:-

⇒ Return truncates value to specified decimal.

Query Syntax:-

```
SELECT TRUNC(45.923, 2) AS T1,
```

```
TRUNC(45.923) AS T2,
```

```
TRUNC(45.923, -1) AS T3.
```

```
FROM DUAL;
```

Output:-

T1

T2

T3

45.92

45

40

MOD Function:-

⇒ Return remainder of division

Ex:- MOD(1600, 100) → 100

Q:- Calculate the remainder of the ratio of salary to commission for all employee whose job title is salesman

⇒ SELECT **ename, sal, comm, MOD(sal, comm) AS remainder**

FROM **emp**

Where **job = 'SALESMAN';**

DATE Function :-

1. MONTHS-BETWEEN() :-

⇒ Returns the Number of months between 2 Dates.

Query Syntax:-

```
SELECT MONTHS-BETWEEN('DATE1', 'DATE2')
FROM DUAL;
```

Ex:-

```
SELECT MONTHS-BETWEEN('1-JAN-23', '1-JAN-24')
FROM DUAL;
```

// Output

2. ADD-MONTHS() :-

⇒ Add months to a given DATE

Query Syntax:-

```
SELECT ADDMONTHS('DATE', N)
FROM DUAL;
```

no. of month
want to add
the Date.

Ex:-

```
SELECT ADDMONTHS('1-JAN-23', 12)
FROM DUAL;
```

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

3. NEXT_DAY():-

⇒ Return NEXT DAY of the date specified.

Query Syntax:-

```
SELECT NEXT_DAY('DATE', 'NAME of the day')  
FROM DUAL;
```

Ex:-

```
SELECT NEXT_DAY('1-JAN-2023', 'FRIDAY')  
FROM DUAL;
```

4. LAST_DAY():-

⇒ Return LAST DAY of the month.

Query Syntax:-

```
SELECT LAST_DAY('DATE')  
FROM DUAL;
```

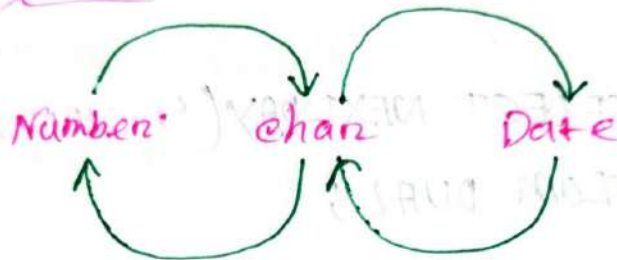
Ex:-

```
SELECT LAST_DAY('1-JAN-23')  
FROM DUAL;
```


Datatype Conversion:-

⇒ Conversion functions that allow us to explicitly convert data from one type to another.

we can convert:-



The Conversion function are:-

1. TO_CHAR()

2. TO_NUMBER()

3. TO_DATE()

#TO_CHAR() function with DATES:-

Query Syntax:-

```

SELECT TO_CHAR(Date, 'fmt')
FROM tablename;
    
```

Rules:-

1. 'fmt' must be enclosed single quotation mark and is case sensitive.

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

1. Is separated from the data value by a comma.

2. Can include any valid date format element.

Date Format model Elements:-

1. YYYY → year in numbers (1989)

2. YEAR → year spelled out

3. MM → 2-digit value for month (03)

4. MONTH → Full Name of the month (march)

5. DY → 3-letter of the day (FRI)

6. DAY → Full name of the day (FRIDAY)

7. DD → Day in No of the month (17)

8. HH24MISS AM → Return the time (208.138.15 AM)

9. HH12MISS AM → Return the time (088.138.15 AM)

10. DD "of" MONTH → Return the concatenation of date and month (10 of march).

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Q: Show the ename, HIREDATE in this (17-MAY-1980) Format.

```
=> SELECT ename, TO_CHAR(HIREDATE, 'DD-MONTH-YYYY')  
FROM emp;
```

// output

Q:- Show the ename, job, HIREDATE in this (Friday-17 of May-1980) Format.

```
=> SELECT ename, job,  
TO_CHAR(HIREDATE, 'DAY-DD "of" MONTH-YYYY')  
FROM emp;
```

// output

Q:- Show your current system time like 8:27:15 AM

```
=> SELECT TO_CHAR(SYSDATE, 'HH12MI:SS AM')  
FROM DUAL;
```

// output

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

TO_CHAR() function with Numbers:-

Query Syntax:-

```
SELECT TO_CHAR(number, 'fmt')  
FROM tableName;
```

Number Format Model:-

- 9 → Represents a Number.(000)
- 0 → Forces a zero to be displayed.(0)
- \$ → Place a Dollar sign (\$000)
- L → Show Local Currency symbol.
- → Print the Decimal point (00.00)
- ,

Ex:- Show the salary of "TURNER" in '\$0,000.00' format

```
⇒ SELECT TO_CHAR(sal, '$0,000.00')  
FROM emp  
WHERE ename IN('TURNER');
```

// Output

TO-Number() function:-

⇒ Convert a character string to a number format using TO-NUMBER().

Query Syntax:-

```
SELECT TO-NUMBER( char, 'fmt')  
FROM tableName;
```

Q:- Show '123' char as a Number.

```
⇒ SELECT TO-NUMBER('123', '999')  
FROM DUAL;
```

TO-DATE() function:-

⇒ Convert a character string to a date format using the TO-DATE() function.

Query Syntax:-

```
SELECT TO-DATE(char-Date, 'fmt')  
FROM tableName;
```

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Q:- Show the Date '1-5-1080' In the formate 01-MAY-1080

⇒ `SELECT TO_DATE('1-5-1080', 'DD-MONTH-YYYY')`
`FROM DUAL;`

// output

#NVL Function:-

⇒ Used to replace NULL, Zero, void, or empty with any value that we want it can be an integer or String.

Rules:-

1. Datatypes that can be used are date, char, and Number.

2. Data types must match

Like:-

`NVL(number, 0)`

`NVL(Date, '01-JAN-1080')`

`NVL(char, 'Any message')`

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Query Statement:-

```
SELECT NVL(comm, 'Default value')  
FROM table-name;
```

Q: Show the Annual Commission of the employees considering all the rows including the null values with their Name.

⇒ Select `ename, NVL(comm, 0) * 12 AS AnnualComm`
FROM emp;

// output

#DECODE() function:-

⇒ Used for Conditional value replacement in a query result. It allows us to compare a value to a set of possible values and return a corresponding result when a match is found.

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Query Syntax:-

```
SELECT DECODE(expression, Search1, result1,  
                Search2, result2,  
                Search3, result3,  
                .... , .... ,  
                default_result)
```

FROM table-Name;

Q:- Show the ename, job, salary of the employees by changing the salary for the people having the job 'CLERK', 'PRESIDENT', 'SALESMAN' respectively with 2%, 5%, and 7% increment as "changed salary"

```
=> SELECT ename, job, sal,  
        DECODE(job, 'CLERK', sal*1.02,  
                'PRESIDENT', sal*1.05,  
                'SALESMAN', sal*1.07, sal) AS  
        "changed salary"
```

FROM EMP;

OUTPUT

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Nesting Function:-

⇒ A function that is contained within another function.

Ex:-

```
SELECT ename, NVL(TO_CHAR(mgr), 'No Manager')  
FROM emp  
WHERE mgr IS NULL;
```

// output

Group Function:-

⇒ Group functions are mathematical functions to operate on sets of rows to give one result per set. It is also called aggregate function.

The types of group function:-

1. AVG() → Calculate Average
2. SUM() → Calculate Sum.
3. COUNT() → Calculate No. of rows in a set.
4. MAX() → Calculating the maximum.
5. MIN() → Calculating the minimum.
6. STDDEV() → Calculating the Standard deviation
7. VARIANCE() → Calculating the Variance.

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Group by function:-

⇒ The group by statement groups rows that have the same values into summary rows, like "Find the number of customers in each country". The GROUP BY statement is often used with aggregate functions { COUNT(), MAX(), MIN(), SUM(), AVG() } to group the result-set by one or more columns.

Query Syntax:-

```
SELECT column-Name, Group-function(column-Name)
FROM Table-Name
WHERE Condition
GROUP BY column-Name
ORDER BY column-Name ;
```

Using MIN & MAX Function:-

Ex:-

```
SELECT MIN(sal), MAX(sal)
FROM emp;
```

// output

#Using AVG & Sum functions:-

Ex:-

```
SELECT AVG(sal), SUM(sal)
FROM emp;
```

#Using Count function:-

Ex:-

```
SELECT COUNT(comm)
FROM emp;
WHERE deptNO=30;
```

// output

#Using Group Function and Null Values Together:-

Ex:-

```
SELECT AVG(comm)
FROM emp;
```

// output

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

-ଏହା (SQL) ର ଏକ ফাংশন - যেখানে Average এর value - if correct
নয় - তবেই ignore করে দেয়। Group functions ignore
- করে null values ফলাফল - এ - ফলাফল থেকে। (যে SQL ফাংশন
NVL function - এখানে Nesting function - যেখানে
- ফলাফল - নামটি - প্রা - ফলাফল - ফলাফল।

```
⇒ SELECT AVG(NVL(comm, 0))  
FROM emp;
```

// output .

Using Group By Clause:-

Ex:-

```
SELECT deptno, AVG(sal)
```

```
FROM emp
```

```
GROUP BY deptno;
```

// output .

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Q:- Show the sum of the salaries for each job except the cases where job is "SALESMAN"

```
⇒ SELECT job, SUM(sal) AS "SUM SALARY"  
FROM emp  
WHERE job NOT IN("SALESMAN")  
GROUP BY job;
```

// output :

Illegal Queries Using Group Function:-

Rules:-

1. We cannot use the where clause to restrict group
2. We use HAVING clause to restrict group.

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Having Clause:-

⇒ It's used if we need to filter the result set based on aggregate functions such as MIN(), MAX(), SUM(), AVG().

Order and Syntax:-

```
SELECT GROUP(Col1)
FROM      tableName
WHERE     Condition
GROUP BY  Col1
HAVING    Condition.
ORDER BY  Col;
```

Ex:-

```
SELECT deptno, MAX(sal)
FROM emp
GROUP BY deptno
HAVING MAX(sal) > 2200;
```

// output →

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Q:- Show the sum of the salaries for each job except the cases where job is "SALESMAN" and the summation is greater than 5000 in a descending order.

```
=> SELECT job, ename, sal, SUM(sal)
      FROM emp
      WHERE job NOT IN('SALESMAN')
      GROUP BY job
      HAVING SUM(sal) > 5000
      ORDER BY job DESC;
```

// output

Q:- Show the maximum Average Salary for each deptno.

```
SELECT MAX(AVG(sal))
      FROM emp
      GROUP BY deptno;
```

// output

Order of Evaluation:-

- 1 SELECT clause
- 2 FROM clause
- 3 WHERE clause
- 4 GROUP BY clause
- 5 HAVING clause
- 6 ORDER clause.

How to Add New Columns:- In our DataBase Table:-

Query Syntax:-

ALTER TABLE table-Name

ADD (column Name1 Datatype(Size),
column Name2 Datatype(size));

Q:- ADD 2 column Nameky ID, CGPA in your emp table.

⇒ ALTER TABLE emp
ADD (ID Number(7),
CGPA Number(1,3));

How to Drop a Column

From a table:-

Query Syntax:-

```
ALTER TABLE table-Name  
DROP COLUMN column-Name
```

Q:- DROP those column which is Added earlier $\Rightarrow$ ALTER TABLE emp

DROP COLUMN ID;

ALTER TABLE emp

DROP COLUMN COTPA;

How to MODIFY a Column

From a table:-

Query Syntax:-

```
ALTER TABLE table-Name  
MODIFY (column-Name DATATYPE(size));
```


TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Q:- Modify Data type of the ID column

⇒ ALTER TABLE emp
MODIFY (ID VARCHAR2(10));

// output

#How to TRUNCATE a table:-

Query Syntax:-

TRUNCATE TABLE table name;

#How to DROP a table:-

Query Syntax:-

DROP TABLE table name;

#DML#

⇒ DML Statement are used for managing data within schema objects.

The operations of DML:-

1. INSERT

2. DELETE

3. UPDATE

4. MERGE

#DELETE DATA FROM

TABLE

Query Syntax:-

1. Remove All Data:-

DELETE FROM table-name;

2. Remove of a specified row's

DELETE FROM table-name
WHERE condition.

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

#How to UPDATE DATA IN A TABLE

Query Syntax:-

1. Updating all rows:-

UPDATE table_Name

SET Column_Name1 = Value,
Column_Name2 = Value;

2. Updating selected records:-

UPDATE table_Name

SET Column_Name1 = Value,
Column_Name2 = Value

WHERE Condition;

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

#How to add Constraint:-

Query Syntax:-

```
ALTER TABLE table-Name  
ADD CONSTRAINT constraint-Name  
CONSTRAINT type (col);
```

Ex:-

```
ALTER TABLE emp  
ADD CONSTRAINT PRK  
PRIMARY KEY (ID);
```

#How to Drop Constraints:-

Query Syntax:-

```
ALTER TABLE table-Name  
DROP CONSTRAINT constraint name;
```

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

HOW TO DISABLE CONSTRAINTS:-

Query Syntax:-

```
ALTER TABLE table-name  
DISABLE constraint-name;
```

Ex:-

```
ALTER TABLE emp  
DISABLE PRIMARY KEY;
```

How to enable constraints:-

Query Syntax:-

```
ALTER TABLE table-name  
ENABLE constraint-name;
```

Ex:-

```
ALTER TABLE emp  
ENABLE PRIMARY KEY;
```

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

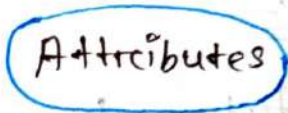
#IP
DIPLOMA

ER Diagram Notation

1. Rectangle  → Represent  sets.

2. Diamonds  → Represent  sets.

3. Lines (—) → Represent the Link Between attributes to entity sets and entity sets to relationship sets.

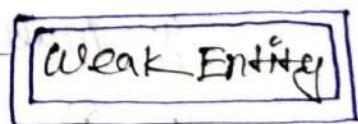
4. Ellipses  → Represent 

there are 2 types of Ellipses:-

(i) Double Ellipses  → Represent Multivalued Attributes

(ii) Dashed Ellipses  → Represent derived Attributes.

5. Underline (—) → Represent primary key Attributes

6. Double Rectangle  → Represent  sets

7. Double Lines (=) → Represent Total participation of an entity in a relationship set.

Entity Set:-

⇒ A entity set is a set of entities that share the same attributes.

Attributes

⇒ An attribute is a property of an entity

Ex:-

Student has attributes like Name, age, ID, GPA

there are six(6) types of attributes:-

1 Simple

2 Composite

3 Single-Valued.

4 Multi-Valued

5 Derived

6 Complex

Simple Attribute:-

⇒ An attribute that cannot be further subdivided into components is a simple attribute.

Ex:-

1. The roll number of a student
2. The ID Number of an employee.

Composite Attribute:-

⇒ An Attribute that can be split into components is a composite attribute.

Ex:-

1. The address can be further split into

i) House number.

ii) Street Number.

iii) City

iv) State

v) Country

vi) pin code.

2. The Name can also split into

i) First Name

ii) Middle Name

iii) Last Name.

Single-valued attribute:-

⇒ The attribute which takes up only a single value for each entity instance is a single-valued attribute.

Ex:-

1. The age of a student

Multi-valued Attribute:-

⇒ The attribute which takes up more than one value for each entity instance is a multi-value attribute.

Ex:-

1. Phone Number of a student

Derived Attribute:-

⇒ An attribute that can be derived from other attributes is a derived attributes.

Ex:-

1. Total marks of a student

2. Average " " " "

TOPIC NAME : _____

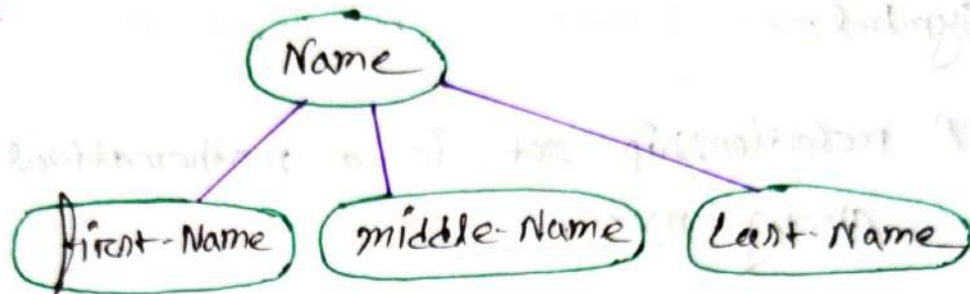
DAY : _____

TIME : _____ DATE : / /

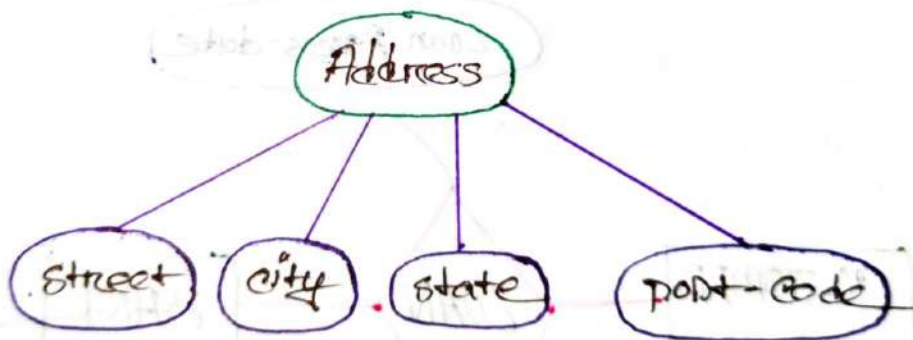
Representation:-

Composite Attribute:-

Ex:1:-



Ex:2:-



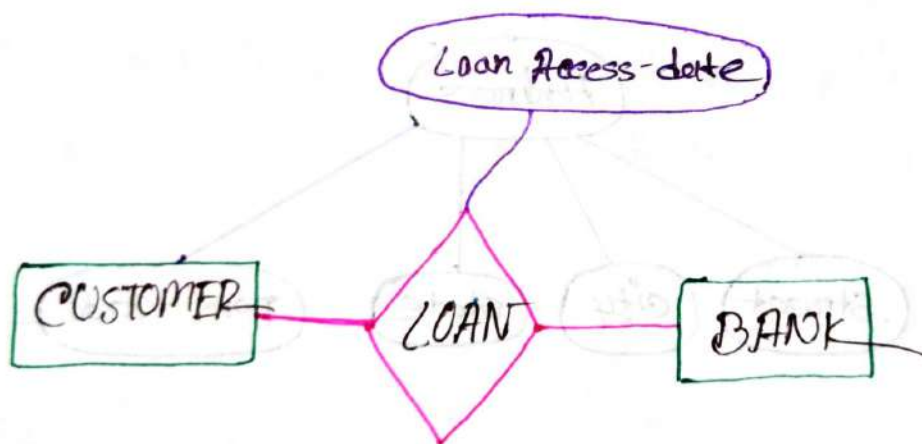
Relationship Sets:-

⇒ In the ER-Diagram the relationship between two strong entity set shown by using a diamond symbol.

→ A relationship set is a mathematical relation among $n \geq 2$.

→ Relationship set may have an attribute.

Ex:-



Degree of a Relationship Sets:-

⇒ It means the Number of entity associated in Relationship set.

- 1. Most of relationship set are binary (2)
- 2. It can be unary, ternary, or n-ary.

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

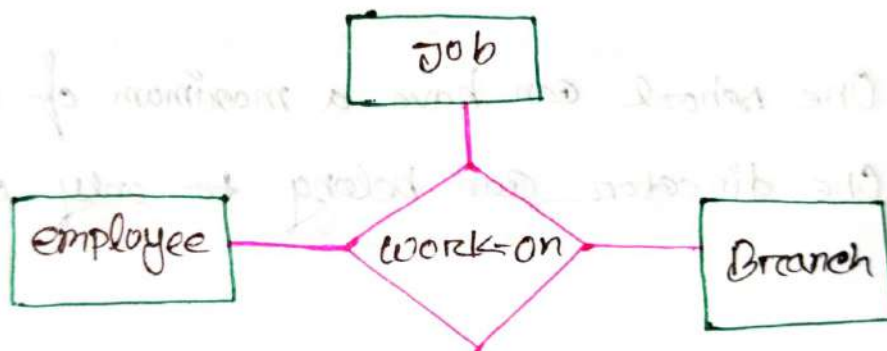
Example 1:-

For Binary:-



Example 2:-

For Ternary:-



Mapping Cardinality:-

⇒ It is the maximum Number of relationship instances in which an entity can participate.

There are 4 types of Cardinality

1. One to one.
2. Many to one.
3. One to many.
4. Many to many.

One to one:-

$\Rightarrow$ One to one cardinality is represented by $(1:1), (\rightarrow)$ symbol. In this, there is at most one relationship from one entity to another entity.

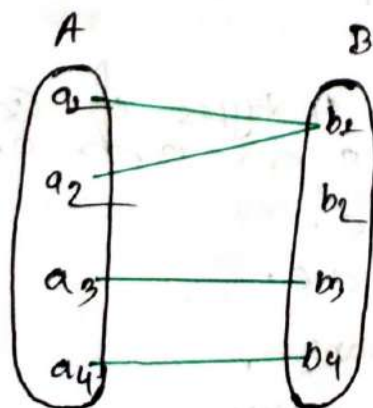
Ex:-

1. One student can have only one student ID, and one student ID belong to only one student.

2. One school can have a maximum of one director, and one director can belong to only one school.

Many to one:-

$\Rightarrow$ In this type of cardinality mapping. An entity in set 'A' is connected to At most one entity in set 'B', represented by $(*8:1), (\leftarrow), (m:1)$ symbol.



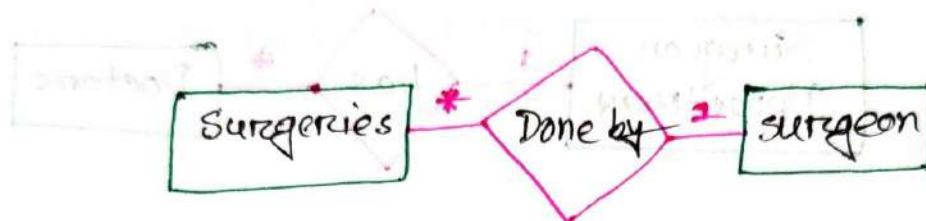
TOPIC NAME : _____

DAY : _____

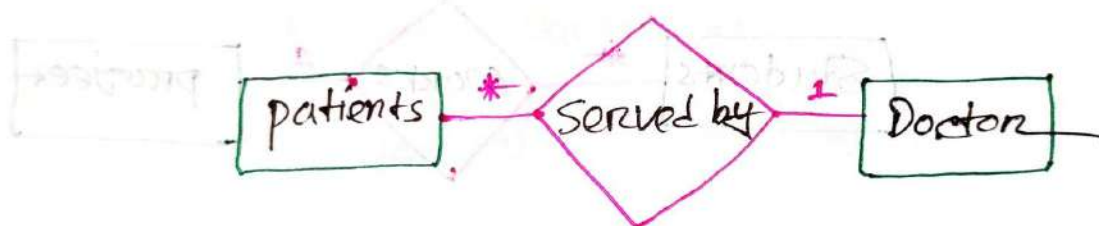
TIME : _____ DATE : / /

Example:-

1. In a particular hospital, multiple surgeries are done by a single surgeon.

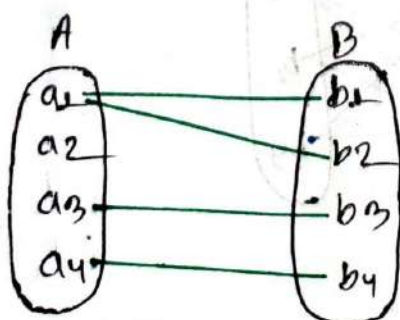


2. There are multiple patients in a hospital who are served by a single doctor.



One to Many

⇒ In this type of cardinality mapping, An entity in set A is associated with any number of entities in set B, represented by ~~(1:*)~~ (1:M)

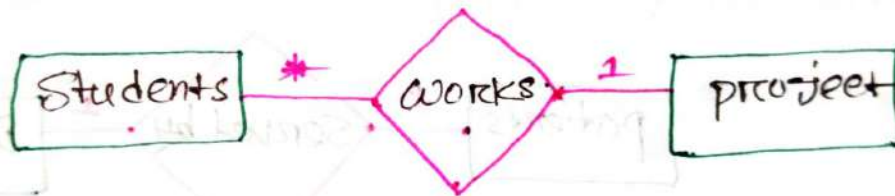


Example:-

- 1 In a particular Hospital, the Surgeon department has multiple doctors

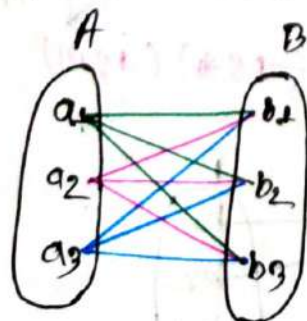


- 2 In a College multiple students can work on a single project.



Many to Many:-

- ⇒ In this type of cardinality mapping, An entity in Set 'A' is Associated with any number of entities in set 'B'. represented by $(*)$ (m:m)



TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Example:-

1. In a particular company, Multiple people work on multiple project.



Participation:-

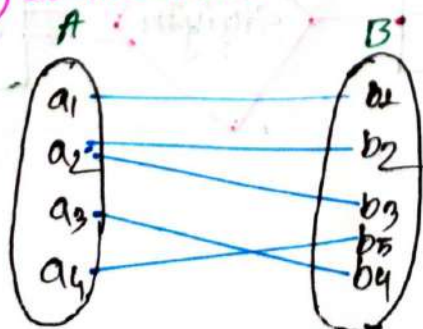
⇒ There are two types of participation.

1. Total Participation

2. Partial Participation.

Total Participation:-

⇒ The participation of an entity set E in a relationship set R is said to be total if every entity in E set participates in at least relationship set in R represent by (=) double line.



TOPIC NAME : _____

DAY : _____

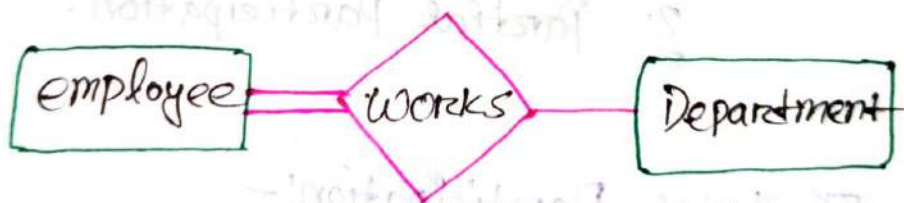
TIME : _____ DATE : / /

⇒ The participation of Entity set 'A' in the relationship set is total, because every entity of SET 'A' participates in the relationship set.

⇒ The participation of Entity set 'B' in the relationship set is also total because every entity of 'B' also participates in the relationship set.

Example -
mon

1. An employee must work for at least one department to exist.

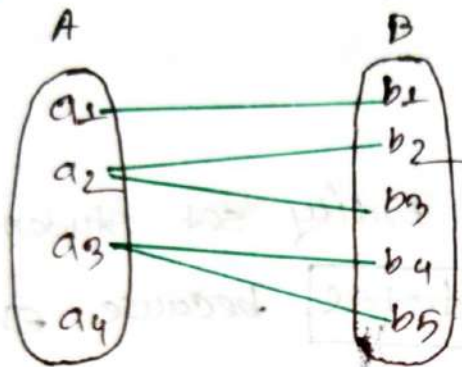


2. Each college must have at least one associated student.



#Partial Participation-

⇒ The participation of an entity set E in relationship set R is said to be **partial** if only some entities in E participate in relationships in R. represented by **(—) Single line.**



→ The participation of entity set A in the relationship set is **partial** because only some entities of A participate in the relationship set.

→ The participation of entity set B in the relationship set is **total** because every entity of B participate in the relationship set.

Ex:-

1. An entity set Student related to entity set Course through Enrolled relationship set



TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

$\Rightarrow$ The participation of Entity set Course in enrolled relationship set is partial because a course may or may not have student enrolled in. It is possible that only some of the course entities are related to the student entity set through the enrolled relationship set.

$\Rightarrow$ The participation of Entity set student in enrolled relationship set is total because every student is expected to relate at least one course through the enrolled relationship set.

Cardinality Limits

$\Rightarrow$ These limits help to define the nature of relationship between entities.

Normally the minimum cardinality could be 0 or 1
the maximum " " " " 1 or *

Common Cardinality Limits (0,1), (1,1), (0,*), (1,*)

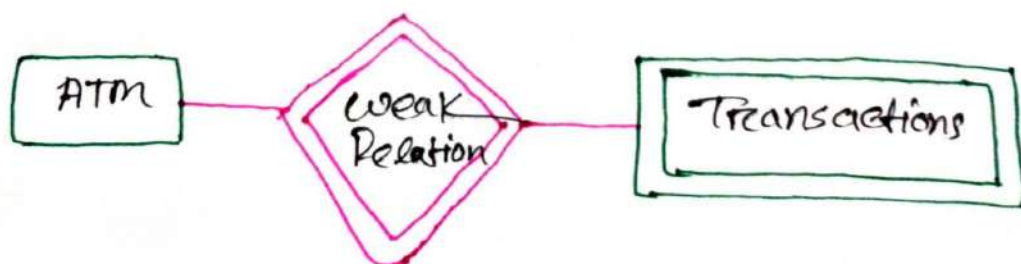


Weak Entities:-

⇒ A weak entity is a type of entity which doesn't have key attribute. It can be identified uniquely by considering the primary key of another entity.

Properties of Weak Entities:-

1. It does not have enough attributes to build a primary key.
2. It represented by a double rectangle symbol.
3. It contains partial key represented by a dashed underline symbol.
4. Member of weak entities is called subordinate entity set.
5. The relationship between one strong and a weak entity set shown by using the double diamond symbol.



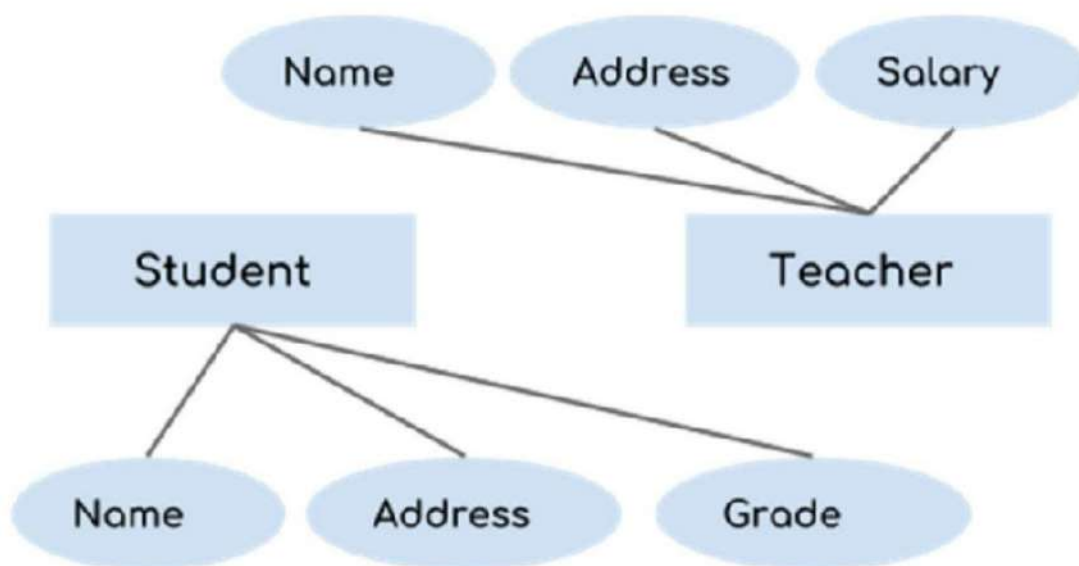
DBMS Generalization

Generalization is a process in which the common attributes of more than one entities form a new entity. This newly formed entity is called generalized entity.

Generalization Example:

- ⇒ Let's say we have two entities Student and Teacher.
- ⇒ Attributes of Entity Student are: Name, Address & Grade
- ⇒ Attributes of Entity Teacher are: Name, Address & Salary

The ER diagram before generalization looks like this:

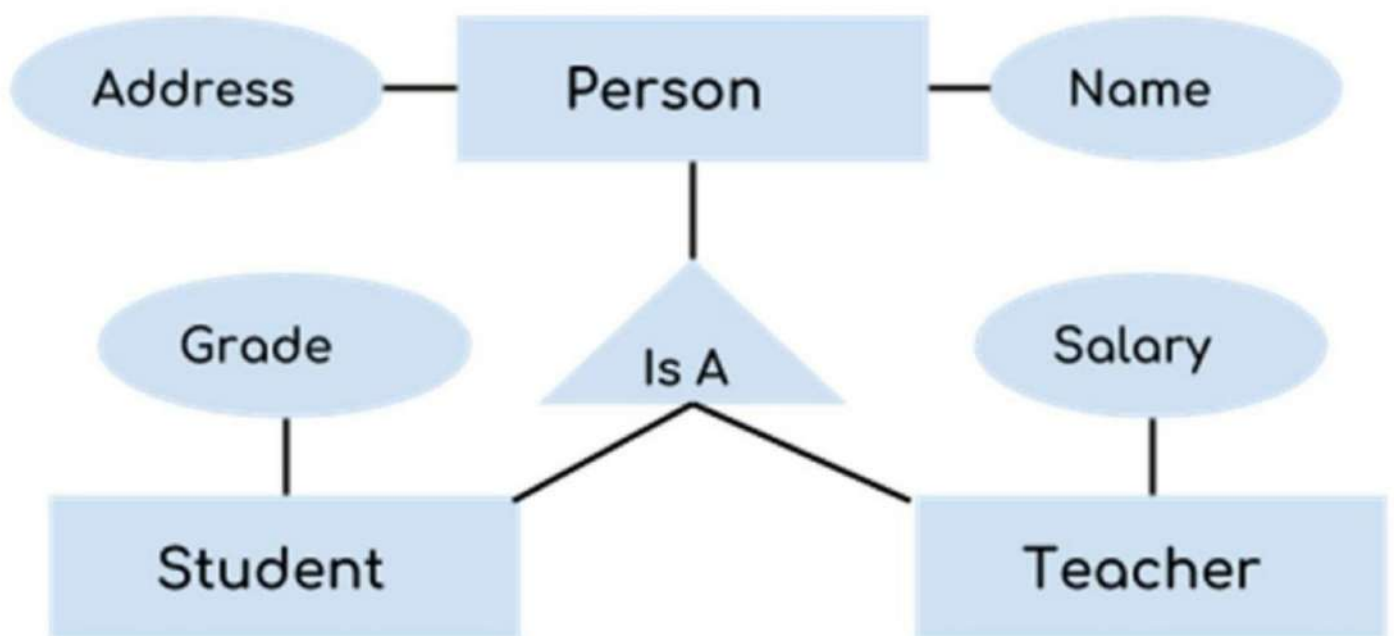


Before Generalization

These two entities have two common attributes: **Name** and **Address**, we can make a generalized entity with these common attributes. Let's have a look at the ER model after generalization.

The ER diagram after generalization:

We have created a new generalized entity Person and this entity has the common attributes of both the entities. As you can see in the following ER diagram that after the generalization process the entities Student and Teacher only has the specialized attributes Grade and Salary respectively and their common attributes (Name & Address) are now associated with a new entity Person which is in the relationship with both the entities (Student & Teacher)

The ER diagram after generalization looks like this:**Generalization****Note:**

1. Generalization uses bottom-up approach where two or more lower level entities combine together to form a higher-level new entity.
2. The new generalized entity can further combine together with lower level entity to create a further higher-level generalized entity.

DBMS Specialization

Specialization is a process in which an entity is divided into sub-entities. You can think of it as a reverse process of generalization, in generalization two entities combine together to form a new higher-level entity. Specialization is a top-down process.

The idea behind Specialization is to find the subsets of entities that have few distinguish attributes. For example – Consider an entity employee which can be further classified as sub entities **Technician, Engineer & Accountant** because these sub entities have some distinguish attributes.

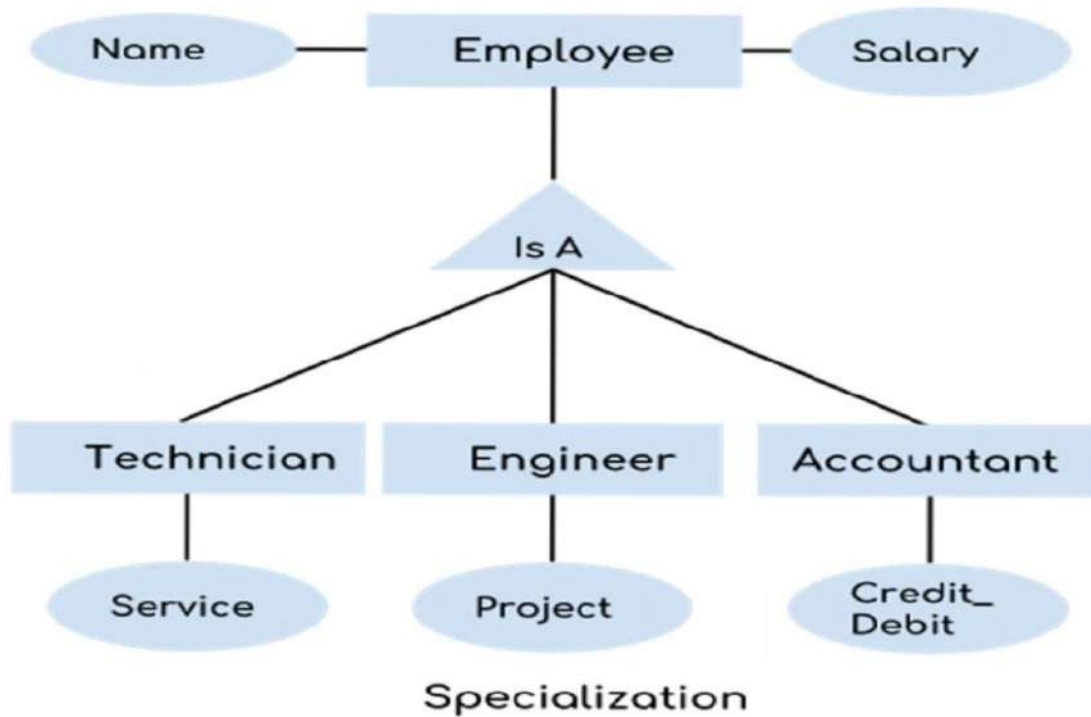
Specialization Example:

In this diagram, we can see that we have a higher-level entity “**Employee**” which we have divided in sub entities “**Technician**”, “**Engineer**” & “**Accountant**”. All of these are just an employee of a company; however, their role is completely different and they have few different attributes. Just for the example, I have shown that Technician handles service requests, Engineer works on a project and Accountant handles the credit & debit details. All of these three employee types have few attributes common such as name & salary which we had left associated with the parent entity “**Employee**” as shown in the above diagram.

TOPIC NAME : _____

DAY : _____

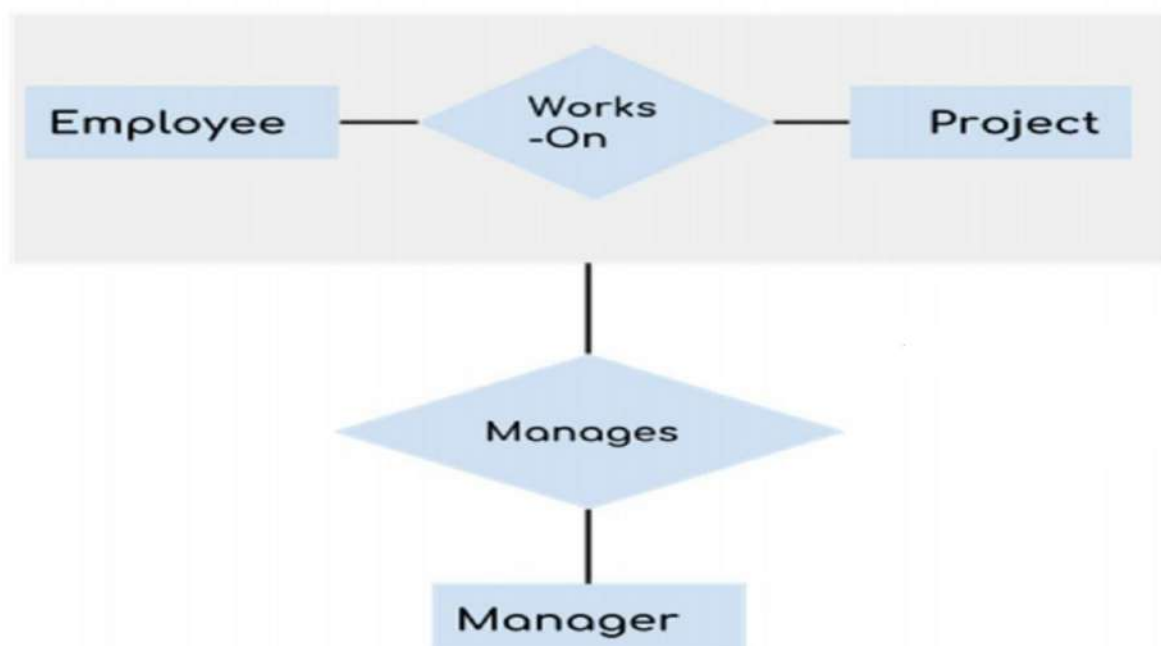
TIME : _____ DATE : / /



DBMS Aggregation

Aggregation is a process in which a single entity alone is not able to make sense in a relationship so the relationship of two entities acts as one entity. I know it sounds confusing but don't worry the example we will take, will clear all the doubts.

Aggregation Example



TOPIC NAME : _____

DAY : _____

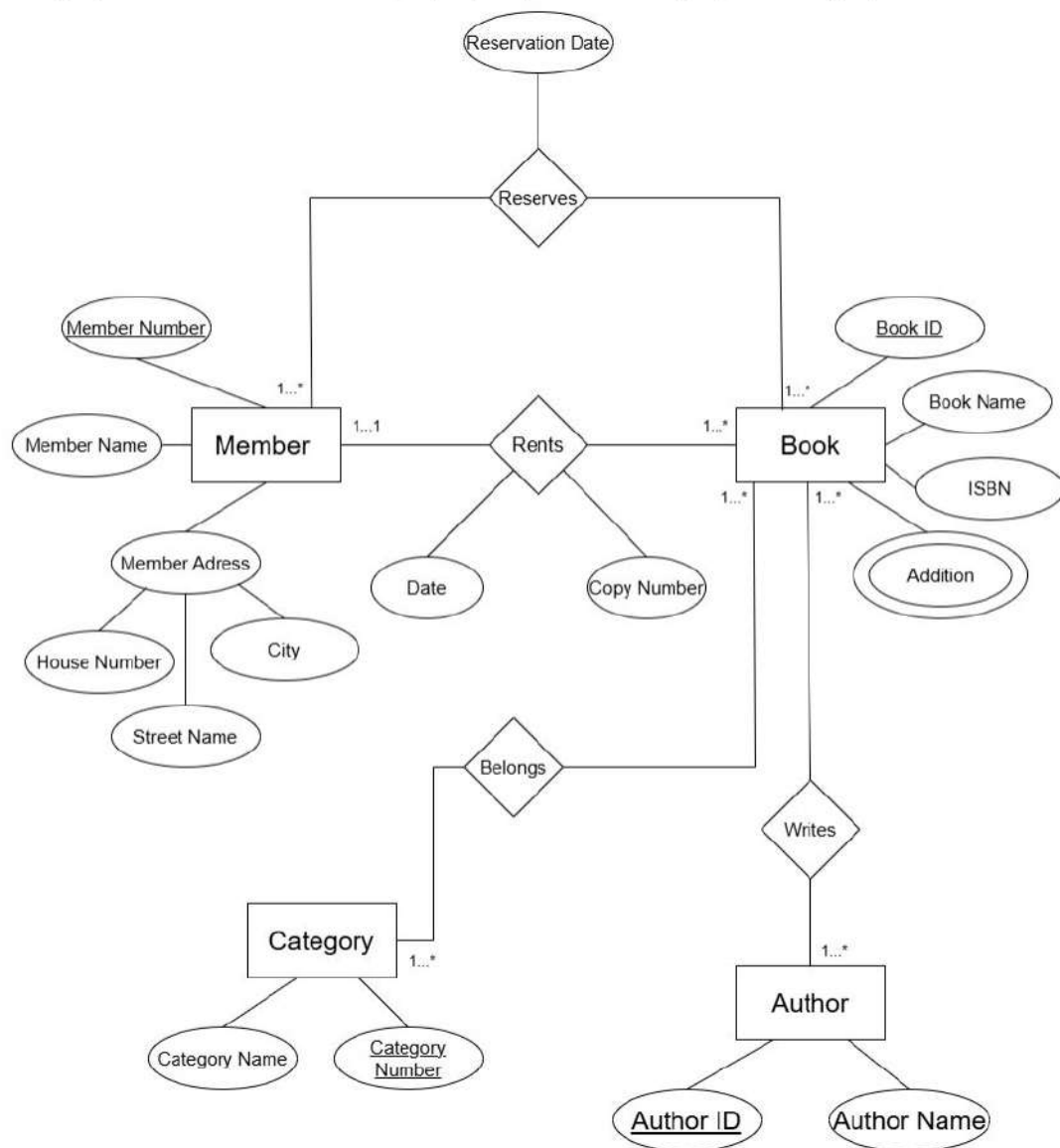
TIME : _____

DATE : / /

In real world, we know that a manager not only manages the employee working under them but he has to manage the project as well. In such scenario if entity "**Manager**" makes a "**manages**" relationship with either "**Employee**" or "**Project**" entity alone then it will not make any sense because he has to manage both. In these cases, the relationship of two entities acts as one entity. In our example, the relationship "**Works-On**" between "**Employee**" & "**Project**" acts as one entity that has a relationship "**Manages**" with the entity "**Manager**".

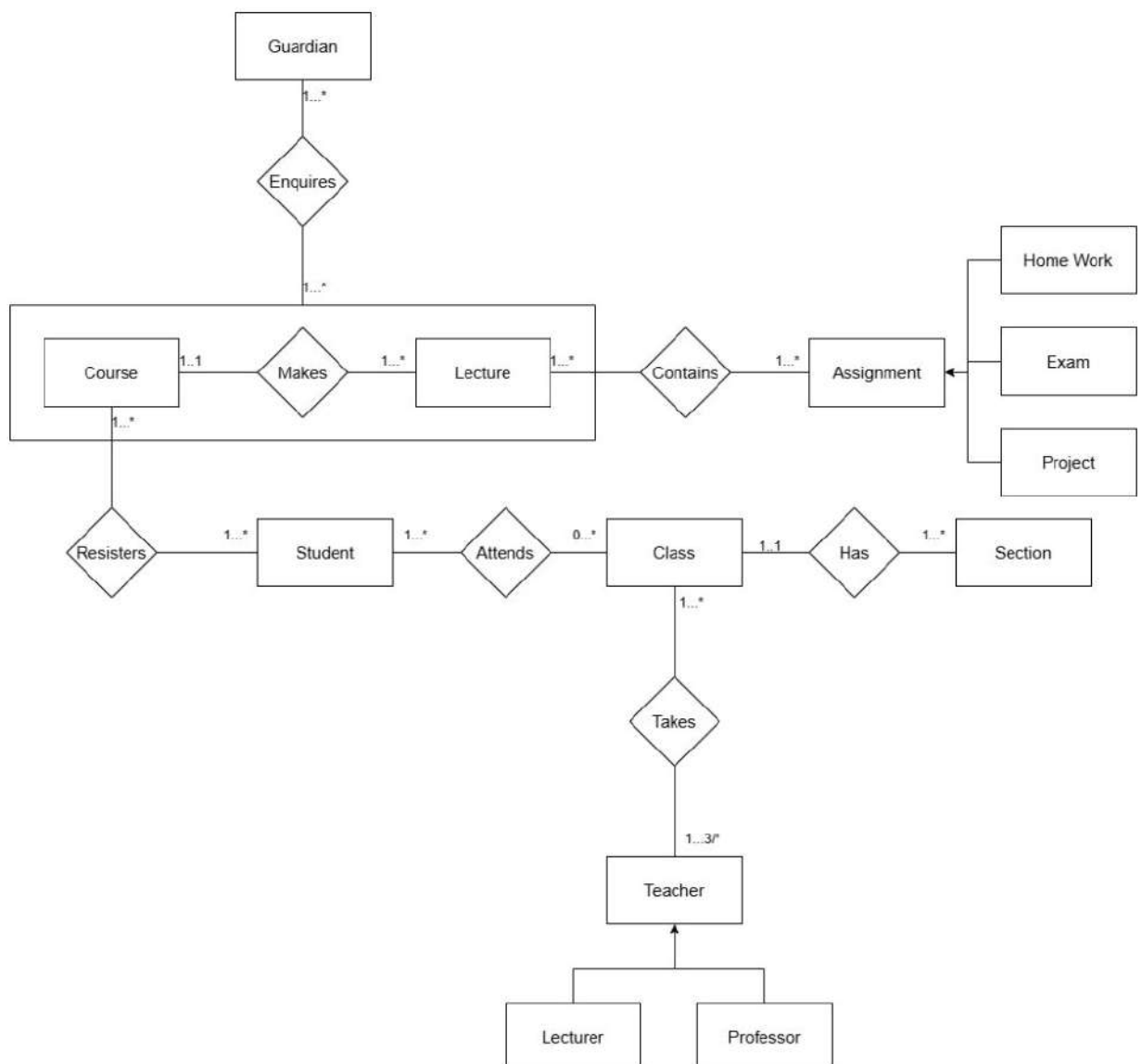
Case Study 01

- ⇒ In a library management system, a member may rent many books. One book may be rented by exactly one member. A member is identified by a member number. The system also stores member name and address. A member address is composed of house number, street name and city. A book is identified by book ID. Book name, ISBN Number and edition of a book are also stored. There may be multiple edition of a book. While borrowing, the date of the borrowing and copy number of the book is stored. A member may also reserve many books. A book may be reserved by many members. To find the priority of the reservation the date of making the reservation is also stored. A book is written by at least one author. An author may write many books but the system stores information of those authors of who has written at least one book stored in the library. To identify an author the system stores author ID along with author name. A book belongs to exactly one category and for a category there must be at least one book. Each category has a name and the unique property of each category is a category number



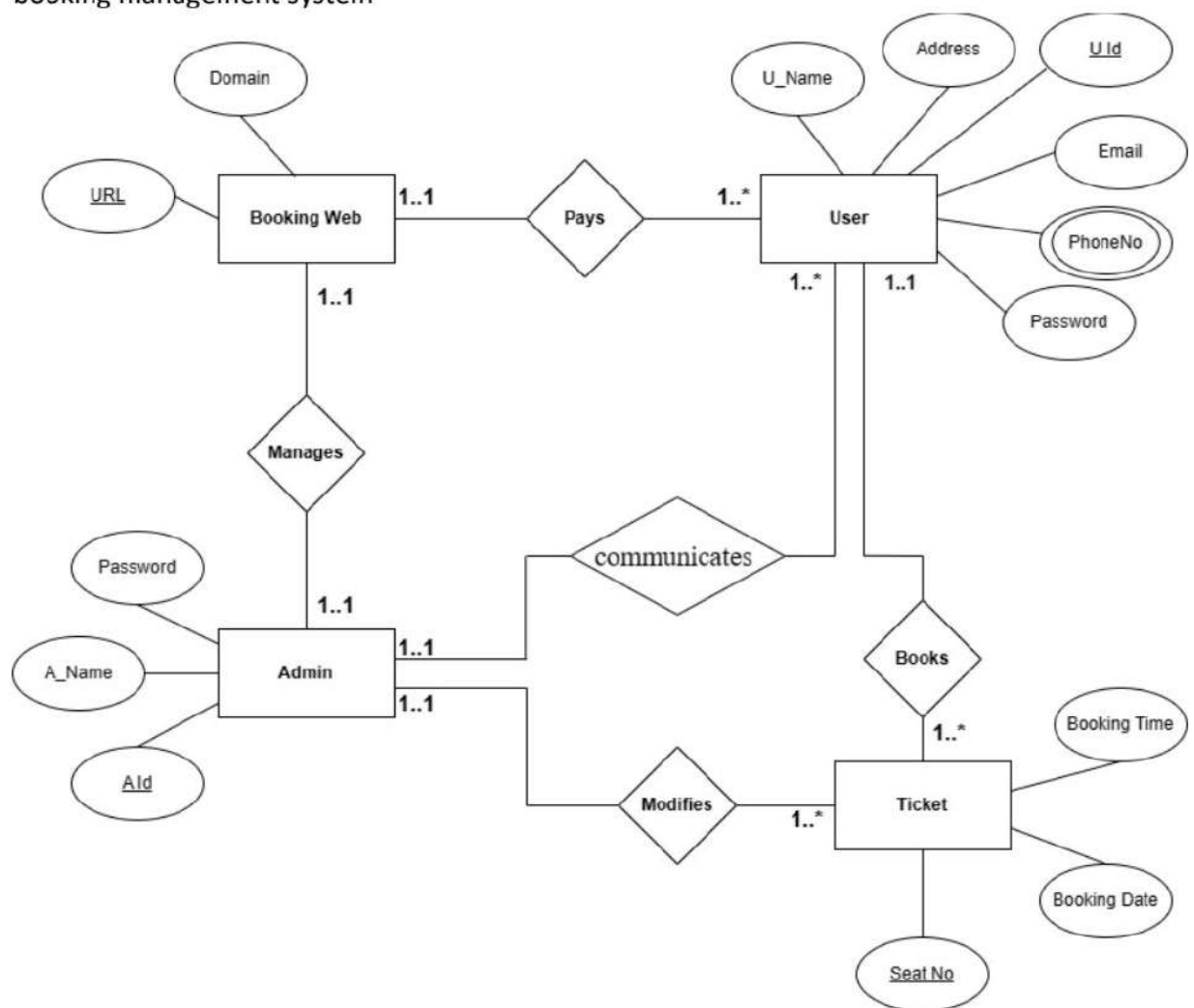
Case Study 02

- ⇒ In a University Management System, a student can attend class. Every class must have a student associated with it via attendance. Every student does not have to be associated with class via attendance. Class has sections. The existence of section is totally dependent on class. Class is taken by teacher. Lecturer and Professor can be generalized as teacher. Student can register for course. Course is made of lectures. Lectures contain assignment. Assignment can be specialized into homework, exam and project. Guardians can come and enquire about course and lecture. Now draw an ER diagram according to the mentioned scenario.



Case Study 03

⇒ In booking management system, The Booking Web serves as the platform where Users need to pay for access. Each Booking Web has a unique URL and domain. booking web can pay multiple users for different purposes. Users interact with the system to booking tickets for various events. User accounts are identified by unique user IDs and include attributes such as usernames, emails, passwords, and phone number, and add. a user can book multiple tickets for various events, and each ticket is booked by one user. Tickets represent individual event bookings and contain details such as seat no, booking date, booking time. Ticket modified by admin. Admin have unique admin IDs, names, and pass. an admin can modify multiple tickets, and each ticket can be modified by one admin. Also, admin communicate with user, many users communicate with one admin but admin communicate one user at a time. Each admin manages one booking web platform, and each booking web platform is managed by one admin. now design an Entity-Relationship (E-R) diagram that visually represents the structure of the proposed booking management system



Case Study 04

⇒ In this system, there are advertisers and publishers. Advertisers buy ads from the publishers. There is an admin who controls the entire system. admin is identified by their admin id and the system also stored their name, phone number, and password. Admin can manage many publisher and Advertisers. Each publisher and Advertisers are managed by exactly one admin. Publishers are identified by their Publishers id and the system also stored Publishers name, Publishers email, Publishers phone number, and Publishers password. Advertisers are identified by their Advertisers id and the system also stored Advertisers name, Advertisers email, Advertisers phone number, Advertisers media, Advertisers media account link, and Advertisers password. Each Advertiser can buy many Ads. Each Ad is associated with many Advertiser. After buying the ads, A advertisers pays bills. An advertiser can pay many bills, but a bill can only be paid by one advertiser. A bill contains the bill number, total price and the payment method. A bill is identified by the bill number. Publishers can upload ads with ad specifications such as type, category, price, and duration. An ad is identified by the ads ID. Each publisher can have many Ads. Each Ad is associated with exactly one publisher. Now design an Entity-Relationship (E-R) diagram that visually represents the structure of the proposed booking management system.

